

MANUAL DE GIT

Guía Completa para Control de Versiones

Generado: 07 de July de 2026

TABLA DE CONTENIDOS

1. Introducción a Git
2. Instalación (macOS, Linux, Windows)
3. Configuración Inicial
4. Conceptos Fundamentales
5. Comandos Básicos
6. Trabajo con Ramas
7. Merge y Resolución de Conflictos
8. Trabajo Remoto y GitHub
9. Flujo de Trabajo GitHub Flow
10. Comandos Avanzados
11. Buenas Prácticas
12. Guía de Troubleshooting
13. Cheat Sheet Rápido

1. INTRODUCCIÓN A GIT

Git es un sistema de control de versiones distribuido que permite:

- Rastrear cambios en tu código a lo largo del tiempo
- Trabajar en paralelo con otros desarrolladores
- Revertir a versiones anteriores si algo sale mal
- Mantener un historial completo del proyecto

Fue creado por Linus Torvalds en 2005 y es el estándar de la industria.

2. INSTALACIÓN

>> macOS

Opción 1: Usando Homebrew (recomendado)

```
brew install git
```

Opción 2: Descargar desde git-scm.com

Opción 3: Instalar Xcode Command Line Tools

>> Linux (Ubuntu/Debian)

```
sudo apt-get update  
sudo apt-get install git
```

>> Linux (Fedora/RHEL)

```
sudo dnf install git
```

>> Windows

Opción 1: Git Bash (recomendado)

Descargar: <https://git-scm.com/download/win>

Opción 2: WSL2 (Windows Subsystem for Linux)

Opción 3: GitHub Desktop (interfaz gráfica)

Descargar: <https://desktop.github.com>

3. CONFIGURACIÓN INICIAL

Antes de usar Git, configura tu identidad:

```
git config --global user.name "Tu Nombre"
git config --global user.email "tu@email.com"
```

Verificar configuración:

```
git config --list
```

4. CONCEPTOS FUNDAMENTALES

>> Repositorio

Carpeta que contiene tu proyecto y su historial completo de versiones.

>> Rama (Branch)

Una línea de desarrollo independiente. La rama por defecto es main o master.

>> Commit

Captura de tus cambios en un momento específico con mensaje descriptivo.

>> Staging Area

Area temporal donde preparas los cambios antes de hacer commit.

>> Remote (Remoto)

Copia del repositorio alojada en un servidor como GitHub, GitLab, etc.

>> Merge (Fusion)

Combinar cambios de una rama a otra.

5. COMANDOS BÁSICOS

>> Crear o Clonar Repositorio

```
git init                # Crear nuevo repositorio
git clone <URL>         # Clonar repositorio remoto
```

>> Ver Estado

```
git status              # Ver archivos modificados
git log                 # Ver historial de commits
git log --oneline       # Log compacto
git diff                # Ver diferencias
```

>> Agregar y Confirmar

```
git add <archivo>      # Agregar archivo específico
git add .               # Agregar todos los cambios
git commit -m "Mensaje" # Hacer commit
```

6. TRABAJO CON RAMAS

>> Listar Ramas

```
git branch          # Ramas locales
git branch -a       # Todas las ramas
git branch -v       # Ramas con último commit
```

>> Crear y Cambiar

```
git branch <nombre>    # Crear rama
git switch <nombre>     # Cambiar a rama
git switch -c <nombre>  # Crear y cambiar
```

>> Eliminar Ramas

```
git branch -d <nombre>    # Eliminar rama local
git push origin --delete <nombre> # Eliminar remota
```

7. MERGE Y RESOLUCIÓN DE CONFLICTOS

>> Mergear Ramas

```
git switch main          # Ir a rama destino
git merge <nombre-rama>  # Mergear rama
```

>> Tipos de Merge

- Fast-forward: Si main no ha cambiado, solo mueve el puntero
- Merge commit: Si ambas ramas han cambiado, crea un merge commit

>> Resolver Conflictos

Si hay conflictos, Git marca secciones con <<< y >>>. Edita el archivo, resuelve, y haz git add + commit.

8. TRABAJO REMOTO Y GITHUB

>> Configurar Remoto

```
git remote add origin <URL>      # Agregar remoto
git remote -v                     # Ver remotos
```

>> Descargar Cambios

```
git fetch origin                  # Descargar sin mergegear
git pull origin <rama>            # Descargar y mergegear
```

>> Subir Cambios

```
git push origin <rama>           # Subir rama
git push -u origin <rama>        # Subir y configurar tracking
```

9. FLUJO GITHUB FLOW

GitHub Flow es el flujo recomendado para trabajo colaborativo:

1. Crea rama: `git switch -c feature/mi-feature`
2. Haz cambios: `git add .` && `git commit -m '...'`
3. Sube rama: `git push -u origin feature/mi-feature`
4. Abre Pull Request en GitHub
5. Otros revisan tu código
6. Mergea el PR
7. Elimina rama remota
8. Actualiza: `git switch main` && `git pull`

10. COMANDOS AVANZADOS

>> Deshacer Cambios

```
git restore <archivo>      # Descartar cambios
git revert <commit>         # Deshacer manteniendo historial
```

>> Stash (Guardado Temporal)

```
git stash                   # Guardar cambios temporalmente
git stash pop               # Recuperar último stash
```

>> Rebase

```
git rebase <rama>          # Reorganizar commits
git rebase -i HEAD~3       # Rebase interactivo
```

>> Cherry-pick

```
git cherry-pick <commit>   # Aplicar commit específico
```

11. BUENAS PRÁCTICAS

>> Mensajes de Commit Claros

Usa presente: 'Agregar validación' no 'Agregué validación'. Se específico y conciso.

>> Commits Pequeños

Un commit = una tarea logica. Evita cambios sin relación en un mismo commit.

>> Ramas por Característica

Crea rama para cada feature: feature/login, fix/bug-123

>> Sincroniza Frecuentemente

Haz pull regularmente para evitar conflictos grandes.

>> No Commitear Secretos

Usa .gitignore para excluir archivos sensibles (.env, tokens, etc.)

12. TROUBLESHOOTING

>> Permission denied al pushear

Configura SSH o usa token de acceso GitHub.

>> No puedo cambiar de rama

Hay cambios sin commitear. Usa 'git stash' temporalmente.

>> Quiero eliminar el último commit

```
git reset --soft HEAD~1      # Mantiene cambios
git reset --hard HEAD~1      # Elimina todo
```

>> Rama eliminada accidentalmente

Usa 'git reflog' para ver historial y recuperar.

>> Conflicto al mergear

Edita archivo, resuelve conflictos, git add, git commit.

13. CHEAT SHEET RÁPIDO

>> CONFIGURACIÓN

```
git config --global user.name 'Nombre'  
git config --global user.email 'email@example.com'  
git config --list
```

>> REPOSITORIO

```
git init  
git clone <URL>  
git status
```

>> CAMBIOS

```
git add <archivo>  
git add .  
git commit -m 'mensaje'  
git diff
```

>> RAMAS

```
git branch  
git switch -c <rama>  
git switch <rama>  
git merge <rama>
```

>> REMOTO

```
git push origin <rama>  
git pull origin <rama>  
git fetch origin
```

RECURSOS ÚTILES

Git Official Documentation:

<https://git-scm.com/doc>

GitHub Learning Lab:

<https://lab.github.com>

Pro Git Book (libre):

<https://git-scm.com/book/es/v2>

Oh Shit, Git!?!:

<https://ohshitgit.com>

Manual generado como parte del proyecto educativo de Git.

Versión 1.0 - 2026